

LiveC Editorial Workflow: Assigning & Deferring Reviewers

Overview

This guide explains how to implement and use the following features in the LiveC editorial workflow:

1. **Assigning multiple reviewers** to a suggestion after review has started (Associate Editor).
2. **Deferring a suggestion to a single reviewer** during triage (Associate Editor).
3. **Reviewer view**: Reviewers can add documentation but cannot edit the section.

1. Assigning Multiple Reviewers (Associate Editor)

Frontend

A. Update `useAssociateEditor.jsx`

Implement the `assign` method to call the backend API. This method should be called when the Associate Editor selects reviewers and submits the assignment form.

```
// ...existing code...
/**
 * Assigns one or more reviewers to a suggestion after review has started.
 * @param {string} suggestionId - The ID of the suggestion.
 * @param {Object} params - Assignment data.
 * @param {string} params.forPrivate - Notes for reviewers (private).
 * @param {string} params.forPublic - Message to the submitter (public).
 * @param {Array<string>} params.reviewers - Array of reviewer IDs.
 */
const assign = async (suggestionId, { forPrivate, forPublic, reviewers }) => {
  try {
    await postAssignReviewers(suggestionId, forPrivate, forPublic, reviewers);
    // Optionally, show a success message or refresh the suggestion view
  } catch (error) {
    console.error(error);
    // Optionally, show an error message to the user
  }
};
// ...existing code...
```

B. Update API Handler

Ensure the API handler matches the backend contract and passes all required fields.

```
// filepath: frontend/src/utils/api-handlers/suggestions/post-reviewers.js
/**
 * Calls the backend to assign reviewers to a suggestion.
 * @param {string} suggestionId
 * @param {string} notes - Private notes for reviewers.
 * @param {string} message - Public message to submitter.
 * @param {Array<string>} reviewers - Reviewer IDs.
 */
export const postAssignReviewers = async (suggestionId, notes, message, reviewers) => {
  try {
    await API.post(`/suggestion/${suggestionId}/assign-reviewers`, { notes, message, reviewers });
  } catch (error) {
    logger.error(error);
  }
};
```

C. UI Integration

- In `FullView.jsx`, add an "Assign Reviewers" button/modal that is visible to Associate Editors after a suggestion has been moved to the "reviewing" state.
- The modal should display a multi-select dropdown of available reviewers (from the `reviewers` array in the hook).
- Include fields for private notes and a public message.
- On submit, call `assign(suggestionId, { forPrivate, forPublic, reviewers })`.
- After assignment, update the UI to reflect the new reviewers (e.g., show them in the suggestion details).

Example Modal Usage:

```
<Button onClick={() => setShowAssignModal(true)} text="Assign Reviewers" />
{showAssignModal && (
  <AssignReviewersModal
    reviewers={reviewers}
    onSubmit={({ selectedReviewers, notes, message }) =>
      assign(suggestionId, { forPrivate: notes, forPublic: message, reviewers: selectedReviewers })
    }
    onClose={() => setShowAssignModal(false)}
  />
)}
```

Backend

- The controller (`post-reviewers.js`) should accept `notes`, `message`, and `reviewers` in the request body.
- The service (`assign-reviewers.js`) should validate the reviewers, update the suggestion, and link the reviewers to the suggestion.
- Ensure the endpoint `/suggestion/:id/assign-reviewers` is correctly wired and returns a success response.

Validation Tips:

- Ensure at least one reviewer is selected.
- Prevent duplicate assignments.

2. Deferring to a Reviewer (Associate Editor, Triage)

Frontend

A. Update `useAssociateEditor.jsx`

Implement the `defer` method. This is called when the Associate Editor chooses to defer a new suggestion to a single reviewer during triage.

```
// ...existing code...
/**
 * Defers a suggestion to a single reviewer during triage.
 * @param {string} suggestionId - The ID of the suggestion.
 * @param {Object} params - Deferral data.
 * @param {string} params.forPrivate - Notes for the reviewer.
 * @param {string} params.forPublic - Message to the submitter.
 * @param {string} paramsReviewer - Reviewer ID.
 */
const defer = async (suggestionId, { forPrivate, forPublic, reviewer }) => {
  try {
    await postDeferral(suggestionId, forPrivate, forPublic, reviewer);
    // Optionally, show a success message or update the suggestion status in the UI
  } catch (error) {
    console.error(error);
    // Optionally, show an error message to the user
  }
};
// ...existing code...
```

B. Update API Handler

```
// filepath: frontend/src/utils/api-handlers/suggestions/post-deferral.js
/**
 * Calls the backend to defer a suggestion to a reviewer.
 * @param {string} suggestionId
 * @param {string} forPrivate - Notes for the reviewer.
 * @param {string} forPublic - Message to the submitter.
 * @param {string} reviewerId - Reviewer ID.
 */
export const postDeferral = async (suggestionId, forPrivate, forPublic, reviewerId) => {
  try {
    await API.post(`/suggestion/${suggestionId}/defer`, { forPrivate, forPublic, reviewer: reviewerId })
  } catch (error) {
    logger.error(error);
  }
};
```

C. UI Integration

- In `TriageView.jsx`, add a "Defer to Reviewer" option as part of the triage actions.
- Show a single-select dropdown of available reviewers.
- Include fields for private notes and a public message.
- On submit, call `defer(suggestionId, { forPrivate, forPublic, reviewer })`.
- After deferral, update the UI to indicate the suggestion is pending reviewer action and disable further AE actions until the review is complete.

Example:

```
<Button onClick={() => setShowDeferModal(true)} text="Defer to Reviewer" />
{showDeferModal && (
  <DeferToReviewerModal
    reviewers={reviewers}
    onSubmit={({ selectedReviewer, notes, message }) =>
      defer(suggestionId, { forPrivate: notes, forPublic: message, reviewer: selectedReviewer })
    }
    onClose={() => setShowDeferModal(false)}
  />
)}
```

Backend

- In `post-deferral.js`, implement the controller to accept `forPrivate`, `forPublic`, and `reviewer` in the request body.
- Call the service to assign the reviewer and update the suggestion status to indicate it is under reviewer consideration.
- Only allow one reviewer for deferral.
- Return a success response.

Example Controller:

```
const postDeferral = async (req, res) => {
  try {
    logger.start('POST Deferral');
    const { id } = req.params;
    const { forPrivate, forPublic, reviewer } = req.body;

    // Only allow one reviewer
    await assignReviewersToSuggestion(id, forPrivate, forPublic, [reviewer]);

    // Optionally update suggestion status to "pending external review" here

    logger.success("suggestion.defer.completed");
    logger.end('POST Deferral');

    return res.status(200).json({ success: true });
  } catch (error) {
    // ...existing error handling...
  }
};
```

Validation Tips:

- Ensure only one reviewer is selected.
- Prevent deferral if the suggestion is not in a triage state.

3. Reviewer View in FullView.jsx

- When a reviewer is logged in, only show the documentation editor.
- Do **not** show the section editor for reviewers.

- Optionally, display a message indicating that section editing is not permitted for reviewers.

```
// filepath: frontend/src/pages/dashboard/suggestion/FullView.jsx
const SectionView = ({ text, id, role, rev }) => {
  // ...existing code...
  {role === Roles.REVIEWER && currentView === 'editor' && <div>Section editing not allowed</div>
  // ...existing code...
}
```

- Ensure the documentation panel is always available for reviewers to add their input.

4. Example: useAssociateEditor.jsx (Excerpt)

```
// filepath: frontend/src/features/associate-editor/useAssociateEditor.jsx
const defer = async (suggestionId, { forPrivate, forPublic, reviewer }) => {
  try {
    await postDeferral(suggestionId, forPrivate, forPublic, reviewer);
  } catch (error) {
    console.error(error);
  }
};

const assign = async (suggestionId, { forPrivate, forPublic, reviewers }) => {
  try {
    await postAssignReviewers(suggestionId, forPrivate, forPublic, reviewers);
  } catch (error) {
    console.error(error);
  }
};
```

5. Summary Table

Functionality	Frontend Location(s)	Backend Location(s)	Key Methods/Endpoints
Assign Reviewers	FullView.jsx, useAssociateEditor.jsx	post-reviewers.js (controller/service)	/suggestion/:id/assign-reviewers
Reviewer View	FullView.jsx	N/A	N/A
Defer to Reviewer	TriageView.jsx, useAssociateEditor.jsx	post-deferral.js (controller/service)	/suggestion/:id/defer

6. Tips

- **Status Management:** Ensure suggestion status is updated correctly in the backend for each action (e.g., "reviewing", "pending reviewer", etc.).
- **Validation:** On backend, validate that only one reviewer is selected for deferral, and multiple for assignment. Prevent duplicate assignments.
- **UI Feedback:** Show confirmation/success messages after actions. Optionally, disable buttons or show loading indicators during API calls.
- **Error Handling:** Display user-friendly error messages if API calls fail.

7. References

- frontend/src/features/associate-editor/useAssociateEditor.jsx
- frontend/src/utils/api-handlers/suggestions/post-reviewers.js
- frontend/src/utils/api-handlers/suggestions/post-deferral.js
- backend/src/controllers/suggestion/post-reviewers.js
- backend/src/controllers/suggestion/post-deferral.js
- backend/src/services/suggestion/assign-reviewers.js

To export as PDF:

Copy this markdown into a `.md` file and use [VS Code Markdown PDF extension](#) or [Dillinger](#) to export as PDF.